

DealMind

A complete, plain-English guide to the project: what it does, how every file and function works, how the pieces talk to each other, and what could be improved.

Project folder: ~/Desktop/dealmind

Size: 7 files, about 230 lines total (Python backend, one HTML page, one JSON data file, config)

Prepared: 29 September 2026

Contents

1. DealMind in one page
2. The big picture (architecture)
3. Tech stack and key concepts
4. File-by-file tour
5. The demo data: data/deals.json
6. The backend: app.py, line by line
7. The frontend: static/index.html
8. What happens when you click each button
9. Resilience: how it survives failures
10. Running and deploying it
11. Issues found and suggested improvements
12. Glossary

1. DealMind in one page

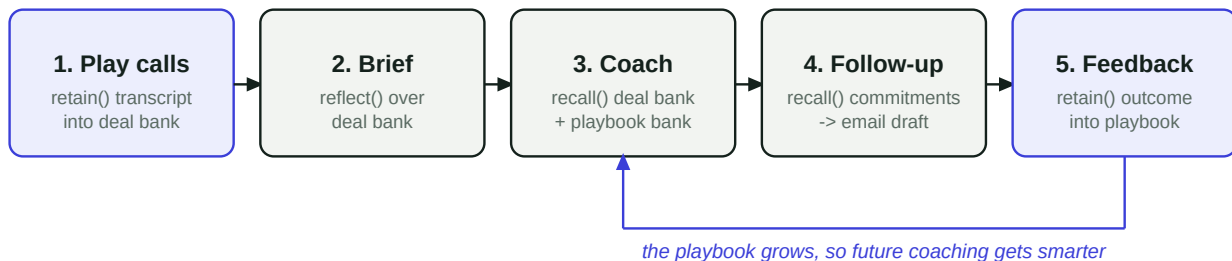
DealMind is a "deal-intelligence agent" for B2B salespeople. Sales deals often span many calls with different people at the customer (an operations lead, a security chief, a CFO). Reps forget details: who objected to what, which numbers were promised, which competitor is in play. DealMind remembers all of it and uses that memory to help the rep.

It is built as a hackathon-style demo around one fictional deal: a vendor selling **Nimbus Agents** (AI agents that automate invoice processing) to a prospect called **Kestrel Logistics**. The rep's name is Arjun. Four sales calls have already happened; the fifth (with the CFO) is about to happen.

What the app lets you do

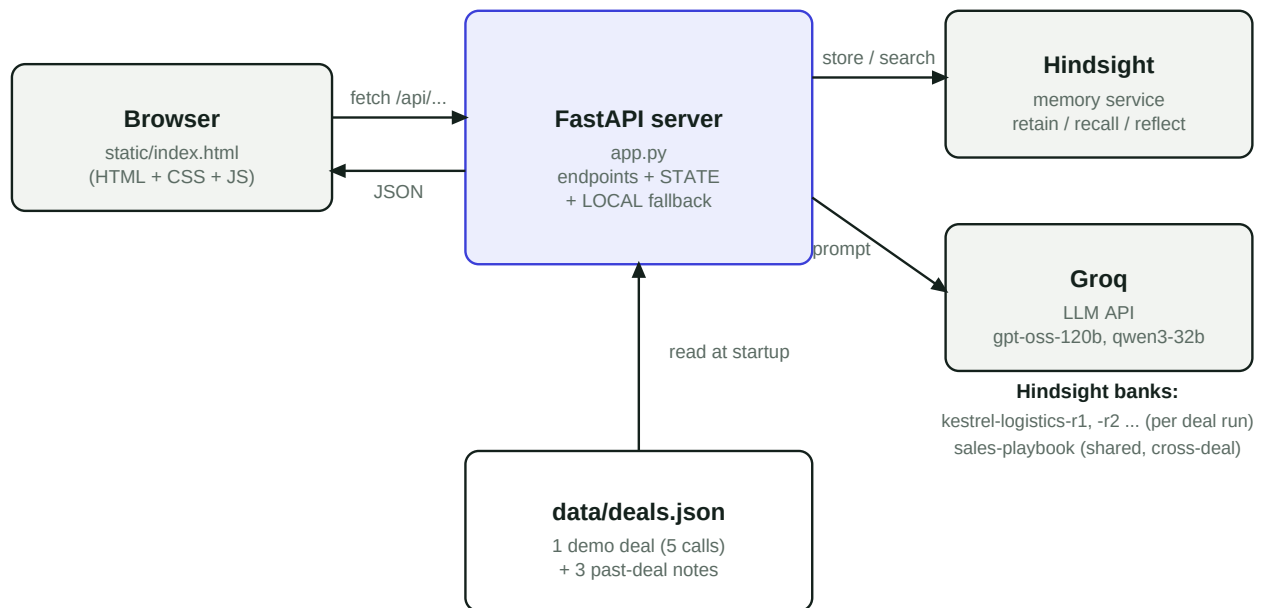
- **Play and remember calls 1-4.** Each call transcript is saved into a long-term memory service called **Hindsight**.
- **Generate a pre-call brief** for call 5, shown side by side: an AI *without* memory (generic, useless) vs. an AI *with* Hindsight memory (specific names, objections, numbers). This comparison is the core of the demo.
- **Get coaching** on an objection (e.g. "CFO wants payback proof"). It pulls from this deal's memory *and* from a shared "sales playbook" of past won and lost deals.
- **Draft a follow-up email** that cites the real commitments and numbers from memory.
- **Teach it:** click "Tactic worked" / "Tactic failed" and the outcome is stored in the shared playbook, so the coaching for future deals improves. This is the "learning loop".

The one-sentence idea: store every conversation in memory, look memory up before every AI action, and feed outcomes back in so the system learns which sales tactics actually work.



2. The big picture (architecture)

DealMind is a small, classic three-tier web app plus two external AI services:



Piece	What it is	Role in DealMind
Browser page	static/index.html (one file with HTML, CSS and JavaScript)	The user interface: list of calls, three tabs (Brief, Coach, Follow-up), buttons.
FastAPI server	app.py (Python)	Serves the page, exposes 8 HTTP endpoints, orchestrates memory + LLM calls, tracks demo state.
Hindsight	External memory service by Vectorize (cloud or self-hosted Docker)	Long-term memory. Stores transcripts as "memories" in "banks", searches them (recall) and can reason over them to produce an answer (reflect).
Groq	External LLM inference API	Runs open large language models very fast. Used to write the coaching, follow-up email, and the no-memory brief.
deals.json	Static demo data	The fictional deal (5 calls) and 3 past-deal post-mortems used to seed the playbook.

Two memory "banks"

Hindsight organizes memory into **banks** (think: separate notebooks). DealMind uses two:

- **Per-deal bank** - named `kestrel-logistics-r1`, `-r2`, and so on. Holds this deal's call transcripts. The `-rN` suffix is a "run" number; clicking "Start a fresh deal" increments it so the demo starts with an empty notebook.
- **Shared playbook bank** - named `sales-playbook`. Holds lessons from past deals (seeded from `deals.json`) plus any feedback you give. It is never reset, so it accumulates knowledge across deals.

3. Tech stack and key concepts

Technology	Where	Why it's used
Python 3.12	Dockerfile	Backend language.
FastAPI	requirements.txt, app.py	Modern Python web framework; each function decorated with <code>@app.get/@app.post</code> becomes an HTTP endpoint. Supports <code>async</code> .
Uvicorn	requirements.txt, Dockerfile	The ASGI server that actually runs the FastAPI app and listens on a port.
Pydantic	app.py (BaseModel)	Validates the JSON body of the feedback request (comes with FastAPI).
hindsight-client	requirements.txt, app.py	Official Python SDK for the Hindsight memory API.
groq	requirements.txt, app.py	Official Python SDK for Groq's OpenAI-compatible chat API.
python-dotenv	app.py (load_dotenv)	Loads the <code>.env</code> file into environment variables. Not listed directly in <code>requirements.txt</code> ; it arrives as a dependency of <code>uvicorn[standard]</code> .
Vanilla JS / CSS	static/index.html	No framework, no build step. Uses <code>fetch()</code> to call the API.
Docker	Dockerfile	Packages the app for deployment on Render/Railway.

Key concepts you need

Memory operations: retain, recall, reflect

Operation	Plain meaning	Used in DealMind for
retain	"Remember this." Save text into a bank. Hindsight extracts facts from it and indexes them.	Saving each call transcript; seeding past deals; saving tactic feedback.
recall	"What do you know about X?" Search a bank and return the most relevant memories.	Coach (searches both banks), Follow-up, and the fallback path for Brief.
reflect	"Think about everything you know and answer this question." Hindsight itself runs an LLM over its memories and returns a written answer.	Main path for the with-memory Brief.

Mission and disposition

When a bank is created, DealMind gives it a **mission** (a description of what this memory is for - tracking stakeholders, objections, requirements, competitors, commitments and winning tactics) and a **disposition** (personality dials: skepticism 4, literalism 3, empathy 3). These steer how Hindsight interprets and reasons over what it stores.

RAG (retrieval-augmented generation)

The Coach and Follow-up features use a pattern called RAG: first *retrieve* relevant memories, then paste them into the LLM prompt so the model *generates* an answer grounded in real facts instead of guessing.

async / await

Almost every backend function is `async`. Calls to Hindsight and Groq are network requests that take seconds; `async` lets the server handle other requests while waiting instead of freezing.

4. File-by-file tour

File	Lines	What it contains
app.py	161	The entire backend: configuration, memory helpers, LLM helper, 8 API endpoints.
static/index.html	42	The entire frontend: layout, styles, and all JavaScript.
data/deals.json	10	Demo data: one deal with 5 calls, plus 3 past-deal post-mortems (the "playbook").
requirements.txt	4	Python packages: fastapi, uvicorn[standard], hindsight-client, groq.
Dockerfile	6	Container recipe for deployment.
README.md	7	Short run/demo/deploy instructions.
.env.example	3	Template for the 3 environment variables. Currently holds the free test keys (see section 11).
.env	3	The actual secrets used locally (HINDSIGHT_URL, HINDSIGHT_API_KEY, GROQ_API_KEY). Not reproduced here.

requirements.txt

```
fastapi
uvicorn[standard]
hindsight-client
groq
```

Dockerfile

```
FROM python:3.12-slim          # small official Python image
WORKDIR /app                  # work inside /app in the container
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # install deps first (cached layer)
COPY . .                      # then copy the code
CMD ["sh", "-c", "uvicorn app:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

Copying requirements.txt before the code is a standard trick: Docker caches the dependency layer, so rebuilding after a code change doesn't reinstall packages. `${PORT:-8000}` uses the PORT variable that hosts like Render/Railway inject, falling back to 8000. Note that `COPY . .` will also copy `.env` into the image, because there is no `.dockerignore`.

README.md (summary)

- Install requirements and set the three env vars.
- Use Hindsight Cloud (with a promo code) or run Hindsight locally via Docker on ports 8888/9999.
- Run `uvicorn app:app --reload` and open `http://localhost:8000`.
- Demo script: play calls 1-4, Generate brief, Coach, Follow-up, then Tactic worked.
- Deploy: push to GitHub, create a Web Service on Render/Railway from the Dockerfile, set the 3 env vars.

5. The demo data: data/deals.json

The JSON has two top-level keys: `deal` (the live deal) and `playbook` (lessons from past deals). The calls are written so that each one reveals a new fact the AI should later remember - it's a carefully designed memory test.

The deal: Kestrel Logistics

Field	Value
id	kestrel-logistics (used to name the memory bank)
company	Kestrel Logistics
product	Nimbus Agents (AI agents for invoice & ops automation)

Cast of characters

Person	Role	What they care about
Arjun	Our sales rep	Closing the deal.
Priya Nair	Ops Director, Kestrel (champion)	4,000 invoices/month, 14 hrs/week lost on exceptions, 2 FTEs + late fees. Dislikes long contracts. Wants pricing clarity.
Rahul Menon	CISO, Kestrel (can veto)	SOC 2 Type II, per-action audit lineage (data, model, approver), India data residency, explainability, live trace, pilot with human review.
Anita Desai	CFO, Kestrel (budget owner)	Payback period; hates big upfront commitments. Joins for the first time on call 5.
Relay AI	Competitor	Slick demo, 1-week setup, 20% cheaper - but can't show per-action audit trails.

The five calls

#	Date	Title	Key facts revealed
1	2026-09-01	Discovery	Volume 4,000 invoices/month; 14 hrs/week on exceptions; cost = 2 FTEs + late fees; CFO Anita owns budget; Arjun promises a usage-based estimate; Priya dislikes long contracts.
2	2026-09-08	Security review	CISO Rahul needs SOC 2 Type II + per-action lineage; data must stay in India; Rahul can veto; Arjun promises lineage docs.
3	2026-09-15	Competitor comparison	Relay AI is 20% cheaper, 1-week setup, but no audit trails; Anita only cares about payback; send numbers for 4,000 invoices.
4	2026-09-22	Explainability objection	Rahul wants a live decision trace and a pilot with human review; a pilot also calms Anita; Arjun offers a 30-day pilot on one queue.
5	2026-09-29	Close attempt (upcoming)	Not held yet. Anita (CFO) joins to decide budget. This is the call the brief prepares for.

The playbook: past deals

Past deal	Outcome	Lesson
Helix Freight	WON	CISO objected on audit trails; a live per-action lineage demo in the security call closed it within a week.

Past deal	Outcome	Lesson
Orbit Retail	LOST	Buyer compared price to a cheaper competitor; rep discounted first and never quantified ROI; lost.
Cedar Health	WON	Skeptical CFO; rep built an ROI calculator from the customer's own invoice volume and offered a 30-day pilot; approved.

Notice the design: each past deal mirrors one Kestrel stakeholder (Helix = Rahul's audit concern, Orbit = the Relay price threat, Cedar = Anita's payback concern). So good coaching should say: *demo live lineage for Rahul, don't lead with a discount against Relay, and give Anita an ROI calc on 4,000 invoices plus a 30-day pilot.*

6. The backend: app.py, line by line

6.1 Imports and configuration (lines 1-26)

```
import os, json, asyncio, pathlib, logging
from fastapi import FastAPI
from hindsight_client import Hindsight
from groq import AsyncGroq
from dotenv import load_dotenv
load_dotenv()                                # read .env into os.environ

BASE = pathlib.Path(__file__).parent          # folder containing app.py
DATA = json.loads((BASE / "data/deals.json").read_text())
DEAL = DATA["deal"]
MODELS = ["openai/gpt-oss-120b", "qwen/qwen3-32b"]    # fallback order
MISSION = ("I am a deal-intelligence memory for an enterprise AI sales team. ...")

HS_URL = os.getenv("HINDSIGHT_URL") or (
    "https://api.hindsight.vectorize.io" if os.getenv("HINDSIGHT_API_KEY") else "http://localhost:8888")
hs = Hindsight(base_url=HS_URL, api_key=os.getenv("HINDSIGHT_API_KEY") or None, timeout=45.0)
llm = AsyncGroq(api_key=os.getenv("GROQ_API_KEY", "missing"))
```

- The deal data is loaded **once at startup** into the DATA/DEAL globals.
- MODELS lists two Groq-hosted models. If the first fails, the second is tried.
- HS_URL logic: use HINDSIGHT_URL if set; otherwise use Hindsight Cloud if an API key exists; otherwise assume a local Docker instance on port 8888.
- Two long-lived clients are created: hs (Hindsight) and llm (Groq). A 45-second timeout protects against hanging memory calls.

6.2 App lifecycle (lines 28-36)

```
@asynccontextmanager
async def lifespan(app):
    try:
        yield                                # app runs here
    finally:
        await hs.aclose()                    # on shutdown, close network clients cleanly
        await llm.close()

app = FastAPI(title="DealMind", lifespan=lifespan)
```

FastAPI's *lifespan* hook runs code at startup (before `yield`) and shutdown (after). Here it only cleans up connections on shutdown.

6.3 Global state (lines 38-42)

```
STATE = {"run": 1, "played": [], "degraded": False, "seeded": False}
LOCAL: dict[str, list[str]] = {}            # fallback store if Hindsight is unreachable

def bank(): return f"{DEAL['id']}-r{STATE['run']}"    # per-deal bank -> "kestrel-logistics-r1"
PLAYBOOK = "sales-playbook"                 # shared cross-deal bank
```

Key	Meaning
run	Demo run counter. Changes the deal bank name so a reset gives an empty memory.
played	Which call numbers have been stored this run (drives the "Remembered" buttons).
degraded	True once any Hindsight call has failed; the UI shows an orange warning banner.
seeded	True once the 3 past deals have been written into the playbook bank.

LOCAL is an in-process dictionary that mirrors everything saved, so if Hindsight is down the app can still show *something*. All of this lives in memory: restarting the server wipes it.

6.4 Helper functions (lines 44-88)

retry(fn, tries=3, base=0.7) - exponential backoff

```
async def retry(fn, tries=3, base=0.7):
    for i in range(tries):
        try: return await fn()
        except Exception as e:
            log.warning("attempt %s failed: %s", i + 1, e)
            if i == tries - 1: raise
            await asyncio.sleep(base * 2 ** i)    # waits 0.7s, then 1.4s
```

Calls a function up to 3 times, waiting longer between each attempt. If all fail, the error is re-raised for the caller to handle. fn is passed as a lambda so a *fresh* request is made each try.

ensure_banks() - create banks and seed the playbook

```
async def ensure_banks():
    for bid in (bank(), PLAYBOOK):
        try: await hs.create_bank(bank_id=bid, name=bid, mission=MISSION,
                                disposition={"skepticism": 4, "literalism": 3, "empathy": 3})
        except Exception as e: log.info("create_bank %s: %s (likely exists)", bid, e)
    if not STATE["seeded"]:
        try:
            await retry(lambda: hs.aretain_batch(bank_id=PLAYBOOK, document_id="past-deals", items=[
                {"content": f"{p['deal']}' ({p['outcome']}) {p['note']}", "context": "closed deal post-mortem"}
                for p in DATA["playbook"]]))
            STATE["seeded"] = True
        except Exception: STATE["degraded"] = True
```

Called before every "play". Tries to create both banks (errors are ignored because the bank probably already exists). The first time, it writes the 3 past deals into the playbook in one batch, e.g. *"Helix Freight (WON): CISO objected on audit trails..."*. Using a fixed document_id="past-deals" means re-seeding should replace rather than duplicate.

remember(bid, text, **kw) - save with local mirror

```
async def remember(bid, text, **kw):
    LOCAL.setdefault(bid, []).append(text)    # always mirror locally
    try: await retry(lambda: hs.aretain(bank_id=bid, content=text, **kw)); return True
    except Exception: STATE["degraded"] = True; return False
```

Always saves a local copy first, then tries Hindsight. Returns True/False for whether Hindsight got it. Extra keyword arguments (context, document_id, timestamp, metadata) are passed straight through to Hindsight.

recall_text(bid, query) - search with graceful fallback

```
async def recall_text(bid, query, max_tokens=2500):
    try:
        r = await retry(lambda: hs.arecall(bank_id=bid, query=query, budget="mid", max_tokens=max_tokens))
        return [f"[{x.type}] {x.text}" for x in r.results]
    except Exception:
        STATE["degraded"] = True
        return [f"[local] {t[:400]}" for t in LOCAL.get(bid, [])[-6:]]    # graceful degradation
```

Asks Hindsight for memories relevant to query, formatting each as [type] text (the type is the kind of fact Hindsight extracted). budget="mid" trades off search depth vs. speed. If Hindsight fails, it returns the last 6 locally-mirrored items, trimmed to 400 characters each - no smart search, but better than nothing.

ask_llm(system, user) - LLM call with model fallback

```
async def ask_llm(system, user):
    last = None
    for m in MODELS:
        for _ in range(2):
            # try gpt-oss-120b, then qwen3-32b
            # 2 tries per model
```

```

try:
    out = await llm.chat.completions.create(model=m, temperature=0.3, max_tokens=700,
        messages=[{"role": "system", "content": system}, {"role": "user", "content": user}])
    txt = (out.choices[0].message.content or "").strip()
    if txt: return txt          # empty answer counts as failure -> retry
except Exception as e: last = e; await asyncio.sleep(0.5)
return f"(LLM unavailable: {last}) Raw memory shown instead."

```

Up to 4 attempts total. **System** message = the instructions/role; **user** message = the data.
 temperature=0.3 keeps answers focused; max_tokens=700 caps length. It never raises - worst
 case it returns an error string. (The message says "Raw memory shown instead", but no endpoint
 actually shows the raw memory in that case.)

6.5 The API endpoints (lines 90-161)

Method + path	Purpose	Uses
GET /	Serve the web page	static/index.html
GET /api/deal	Company, product, state, and the list of calls	DEAL, STATE
POST /api/reset	Start a fresh demo run (new empty deal bank)	STATE
POST /api/play/{n}	Store call n (1-4) in memory	ensure_banks, remember (retain)
GET /api/brief?memory=0 1	Pre-call brief without / with memory	ask_llm, or reflect (fallback: recall + ask_llm)
GET /api/coach?objection=...	Tactics for handling an objection	recall on both banks + ask_llm
GET /api/followup	Draft a follow-up email	recall + ask_llm
POST /api/feedback	Record whether a tactic worked	remember into playbook
GET /api/memories	List up to 30 memories from the deal bank (debug; not used by UI)	list_memories

GET /api/deal

Returns {company, product, state, calls:[{n,title,date,transcript}]}. The page calls this on load and after every action to refresh itself.

POST /api/reset

STATE.update(run=run+1, played=[], degraded=False). Bumping the run number means bank() now returns e.g. kestrel-logistics-r2, a brand-new empty bank. Old banks remain in Hindsight; nothing is deleted. The playbook is untouched, by design.

POST /api/play/{n}

```
await ensure_banks()
c = next((c for c in DEAL["calls"] if c["n"] == n and n < 5), None)
if not c: return {"ok": False, "error": "Call 5 is the upcoming call - use Brief."}
ok = await remember(bank(), f"Call {n} - {c['title']} with {DEAL['company']} on {c['date']}\n{c['transcript']}",
                    context="sales call transcript", document_id=f"call-{n}",
                    timestamp=datetime.fromisoformat(c["date"]), metadata={"deal": DEAL["id"], "call": str(n)})
```

Finds the call (rejecting call 5 because it hasn't happened). The transcript is prefixed with a header line and stored with rich metadata: a **context** label, a **document_id** (so replaying a call replaces rather than duplicates), the real call **timestamp** (so Hindsight can reason about time order), and tags.

GET /api/brief - the headline comparison

```
if not memory or not STATE["played"]:
    return {"text": await ask_llm("You are a sales-prep assistant. 5 bullets max.",
    "Brief me for my next call with Kestrel Logistics. I have no notes."), "memory": False}
try:
    result = await retry(lambda: hs.areflect(bank_id=bank(), query=BRIEF_Q, budget="mid",
    context="rep is about to join call 5 with the CFO; answer as a concise bulleted brief"))
    return {"text": result.text, "memory": True}
except Exception:
    mem = "\n".join(await recall_text(bank(), BRIEF_Q))
    return {"text": await ask_llm("...use only the memory.", mem), "memory": True, "fallback": True}
```

- **memory=0** (left panel): the LLM is asked with no context. It can only produce generic advice - this is the "before" picture.
- **memory=1** (right panel): Hindsight's **reflect** reasons over all stored calls to answer BRIEF_Q (stakeholders, objections, requirements, competitor, commitments, budget owner).

- If reflect fails, it falls back to **recall** + Groq, and flags `fallback: true` so the UI shows the warning banner.

GET /api/coach

Runs two recalls with the objection text as the query - one on this deal's bank, one on the playbook bank - and gives both to the LLM with instructions to recommend 2-3 tactics, **cite the source deal or call** for each, and say what NOT to do if a past deal was lost that way. It also returns the raw memories as sources (the UI doesn't display them yet). This is where cross-deal learning shows up.

GET /api/followup

Recalls "commitments we made, requirements, stakeholder names, next steps" and asks the LLM to write a short, no-fluff email from Arjun to Priya referencing specific numbers and names.

POST /api/feedback - the learning loop

```
class Feedback(BaseModel):
    tactic: str
    worked: bool

ok = await remember(PLAYBOOK, f"{DEAL['company']} ({'WON' if f.worked else 'LOST'}): tactic '{f.tactic}' "
    f"{'worked' if f.worked else 'did not work'} with the CFO.", context="tactic outcome")
```

Pydantic validates the JSON body. The outcome is written into the **shared playbook** in the same "Deal (WON/LOST): ..." style as the seeded past deals, so future Coach calls will retrieve it.

GET /api/memories

Debug helper that lists up to 30 memories in the current deal bank. It defensively uses `getattr` for several possible field names because the SDK's response shape wasn't certain. Handy for checking what Hindsight extracted, e.g. by opening `/api/memories` in the browser.

7. The frontend: static/index.html

One self-contained file: no React, no build tools. It has three parts.

7.1 Styles (lines 2-16)

- CSS variables in `:root` define the palette: pale green-grey background, dark ink, indigo accent (`#3b3fd8`) for "good/with memory", orange (`#c2410c`) for "bad/warning".
- Layout is a two-column CSS grid (320px call list + flexible main area) that collapses to one column under 800px wide, so it works on phones.
- `.cmp` is the side-by-side comparison grid for the brief; `.banner` is the hidden orange "Hindsight is unreachable" warning.
- Accessibility touch: `button:focus-visible` draws a clear outline for keyboard users.

7.2 Markup (lines 18-22)

- **Header:** title "DealMind" and a subtitle that is replaced with "Kestrel Logistics - Nimbus Agents...".
- **Left card:** "Calls with the prospect" list (filled by JS) and a "Start a fresh deal" button.
- **Right section:** the warning banner, three tab buttons (Pre-call brief, Coach, Follow-up email), and an empty `#pane` that JS fills based on the selected tab.

7.3 JavaScript (lines 23-42)

Function	What it does
<code>\$ / api</code>	Tiny helpers: <code>\$('#x')</code> = <code>querySelector</code> ; <code>api(url, opts)</code> = <code>fetch</code> + <code>parse JSON</code> .
<code>load()</code>	GET <code>/api/deal</code> , update subtitle, show/hide the degraded banner, draw the call list (buttons show "Play and remember" or a disabled "Remembered"; call 5 says "The call you are about to join"), then <code>render()</code> .
<code>play(n, btn)</code>	Shows "Retaining...", POSTs <code>/api/play/n</code> , reloads.
<code>reset()</code>	POSTs <code>/api/reset</code> , reloads.
tab click handler	Highlights the clicked tab, stores it in <code>tab</code> , calls <code>render()</code> .
<code>render()</code>	Writes the HTML for the current tab into <code>#pane</code> . Brief tab: button + two panels ("Without memory" / "With Hindsight memory (N calls retained)"). Coach tab: objection text box, button, output, and Tactic worked/failed buttons. Follow-up tab: button + output.
<code>runBrief()</code>	Fires both <code>/api/brief?memory=0</code> and <code>?memory=1</code> in parallel and fills each panel as it returns.
<code>runCoach()</code>	Calls <code>/api/coach</code> with the text box value (URL-encoded).
<code>runFu()</code>	Calls <code>/api/followup</code> .
<code>fb(worked)</code>	POSTs <code>/api/feedback</code> with <code>tactic = "Response to: <objection text>"</code> and shows a browser alert.

8. What happens when you click each button

Opening the page

Browser requests / -> FastAPI returns index.html -> JS calls /api/deal -> the call list appears.

"Play and remember" on call 2

- JS: POST /api/play/2.
- Server: ensure_banks() creates kestrel-logistics-r1 and sales-playbook if needed and seeds the 3 past deals (first time only).
- Server: remember() mirrors the transcript to LOCAL and retains it in Hindsight with date + metadata. Hindsight extracts facts such as "Rahul Menon is the CISO", "data must stay in India".
- Server adds 2 to STATE.played; JS reloads and the button now says "Remembered".

"Generate brief for call 5"

- Left panel: Groq writes a generic brief with no knowledge of Kestrel.
- Right panel: Hindsight reflect produces something like: *Anita Desai (CFO, budget owner) cares about payback and hates upfront commitments; Rahul (CISO, veto) needs per-action lineage, SOC 2, India residency, a live trace; competitor Relay AI is 20% cheaper but lacks audit trails; we promised a usage-based estimate, lineage docs, and a 30-day pilot.*

"Get coaching"

- Recall from deal bank (e.g. Anita's payback concern, the 30-day pilot) + recall from playbook (Cedar Health WON with ROI calc + pilot; Orbit Retail LOST by discounting).
- LLM combines them: e.g. build an ROI calculator on 4,000 invoices (from Cedar), lead with the 30-day pilot, and do NOT open with a discount to match Relay (from Orbit).

"Draft follow-up"

Recall commitments/names/next steps -> LLM writes an email from Arjun to Priya citing 4,000 invoices, 14 hours/week, lineage docs, the 30-day pilot, etc.

"Tactic worked" / "Tactic failed"

Stores e.g. *"Kestrel Logistics (WON): tactic 'Response to: CFO wants payback proof...' worked with the CFO."* in the playbook. After "Start a fresh deal", that lesson is still there for the next deal's coaching.

"Start a fresh deal"

Run number goes up, played list clears, UI shows all calls unplayed; the new deal bank is empty but the playbook remembers everything learned.

9. Resilience: how it survives failures

For a live demo, the author built in several layers of protection so the app never just crashes on stage:

Failure	What DealMind does
Hindsight slow / flaky	retry() with exponential backoff (3 tries); 45 s client timeout.
Hindsight fully down	remember() still saves to LOCAL; recall_text() returns last 6 local items; brief falls back from reflect to recall+LLM; STATE.degraded = true and the UI shows an orange banner.

Failure	What DealMind does
Bank already exists	create_bank errors are caught and logged as "likely exists".
Groq model down or returns empty	ask_llm tries each of 2 models twice; returns a readable error string instead of raising.
Replaying a call	document_id="call-n" lets Hindsight treat it as the same document instead of a duplicate.

10. Running and deploying it

Locally

```
cd ~/Desktop/dealmind
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# make sure .env has HINDSIGHT_URL, HINDSIGHT_API_KEY, GROQ_API_KEY
uvicorn app:app --reload
# open http://localhost:8000
```

Environment variables

Variable	Purpose
HINDSIGHT_URL	Hindsight endpoint: cloud (https://api.hindsight.vectorize.io) or local Docker (http://localhost:8888).
HINDSIGHT_API_KEY	Key for Hindsight Cloud. Not needed for local Docker.
GROQ_API_KEY	Key for Groq's LLM API (get one at console.groq.com).

Running Hindsight yourself (optional)

```
docker run -p 8888:8888 -p 9999:9999 ghcr.io/vectorize-io/hindsight
```

Port 8888 is the API; 9999 is its web UI. Hindsight itself needs an LLM key configured per its docs.

Deploying

Push to GitHub, create a "Web Service" on Render or Railway using the Dockerfile, set the three environment variables in the host's dashboard. The host injects PORT, which the Dockerfile's CMD respects.

11. Issues found and suggested improvements

The code is compact and thoughtfully built for a demo. These are the things worth knowing or fixing, most important first.

Housekeeping for later

Test API keys in .env.example. The template holds the free test keys (Hindsight hsk_..., Groq gsk_...) the project is currently using. That's fine while testing. Before the repo goes public, swap them for placeholders like `your-key-here` and add a `.gitignore` with `.env`. Even free keys are tied to an account, and if someone else uses them they eat into its rate limits and quota.

.env ends up in the Docker image. There's no `.dockerignore`, so `COPY . .` copies the keys file into the image. Add a `.dockerignore` with `.env`. The host's environment variables are the intended place for keys.

Behaviour / design limitations

- **State lives in memory.** STATE and LOCAL vanish on restart. The run counter goes back to 1, so after a restart the app reuses bank `kestrel-logistics-r1`, which may already contain old memories - the "with memory" brief could reflect calls the UI shows as unplayed. Persisting the run number (or using a timestamp) fixes this.
- **Single user.** Everyone who opens the deployed site shares one STATE. Two people demoing at once will interfere.
- **Feedback records the objection, not the tactic.** The UI sends "Response to: <objection>" rather than what the coach actually recommended, so the playbook learns less than it could. Sending the coaching text (or letting the user pick the tactic) would make the loop meaningful.
- **Hard-coded names.** "Kestrel Logistics", "Arjun", "Priya" and "the CFO" are baked into prompts, so the app only works for this one deal. They could be read from `deals.json`.
- **Coach/Follow-up before any call is played** don't call `ensure_banks()`, so recall hits a bank that may not exist, which trips the degraded banner.
- **Coach sources aren't displayed** even though the API returns them - showing them would make the "cites past deals" claim visible.
- The fallback message "Raw memory shown instead" isn't true - no endpoint shows raw memory in that case.
- The frontend inserts data with `innerHTML`. Safe today because data is local, but would be an XSS risk if deal data ever came from users.
- Uses `alert()` after feedback; an inline confirmation would be smoother.
- `python-dotenv` is imported directly but only installed indirectly (via `uvicorn[standard]`); listing it explicitly in `requirements.txt` would be safer.
- No tests, no authentication, and no rate limiting on endpoints that use the LLM/memory quota.

Ideas to extend it

- Multiple deals with a deal picker; upload real call transcripts (or connect to Zoom/Gong).
- Show which memories were used under each answer (sources panel).

- A "memories" tab using the existing /api/memories endpoint to visualize what Hindsight extracted.
- Persist state in a small database (SQLite) and support multiple users/sessions.

12. Glossary

Term	Meaning
Agent	Software that uses an LLM to take actions or produce outputs toward a goal, often with memory and tools.
API / endpoint	A URL on the server the browser can call to get data or trigger an action.
ASGI / Uvicorn	The standard interface and server that run async Python web apps like FastAPI.
Bank	A separate, named memory store in Hindsight.
Budget (recall/reflect)	How much effort Hindsight spends searching/reasoning (low/mid/high).
CFO / CISO	Chief Financial Officer (controls budget) / Chief Information Security Officer (controls security approval).
Champion	The person at the customer who wants the deal to happen (here, Priya).
Degraded mode	Running with reduced features because a dependency is down.
Exponential backoff	Retrying with growing waits (0.7 s, 1.4 s, ...) to avoid hammering a failing service.
Groq	A company offering very fast LLM inference via an OpenAI-compatible API.
Hindsight	An agent-memory system by Vectorize: retain facts, recall them, reflect over them.
LLM	Large language model, e.g. gpt-oss-120b or Qwen3-32B.
Payback period	How long until a purchase's savings cover its cost - what CFOs care about.
Pilot	A small, time-limited trial before a full commitment.
Playbook	A team's collected knowledge of which sales tactics work.
RAG	Retrieval-augmented generation: fetch relevant facts, then have the LLM write using them.
SOC 2 Type II	An independent audit showing a vendor's security controls work over time.
System prompt	Instructions that set the LLM's role and rules for a request.
Temperature	LLM randomness setting; 0.3 = fairly focused and consistent.